



摘要

分子动力学 (Molecular Dynamics, MD) 模拟是研究生物大分子动态行为的重要计算工具, 被誉为“计算显微镜”。随着全细胞模拟目标的提出, 传统基于快速傅里叶变换 (Fast Fourier Transform, FFT) 的长程静电求解方法——如粒子网格 Ewald (Particle Mesh Ewald, PME) 和粒子-粒子-粒子-网格 (Particle-Particle Particle-Mesh, PPPM) ——在大规模并行计算中面临严重的通信瓶颈。本研究针对这一关键问题, 提出了一种基于高斯和分解 (Sum-of-Gaussians, SOG) 与随机批量采样 (Random Batch Sampling) 相结合的新型长程静电求解框架, 即基于随机批量采样与高斯和分解的长程静电方法 (Random Batch Sum-of-Gaussians, RBSOG)。

本研究构建了面向膜体系的分子动力学模拟原型系统, 实现了三种长程静电求解器: 直接求和参考解 ($O(N^2)$)、PPPM 基线方法和 RBSOG 方法。通过系统化的基准测试, 从计算性能、统计方差和结构稳定性三个维度对 PPPM 和 RBSOG 进行了全面比较。基于本文重新串行执行的真实实验结果, RBSOG 在当前纯 Python 原型中的平均每步时间显著高于 PPPM, 在 10 个随机种子统计下约为其 5.47 倍; 同时, RBSOG 的压力方差均值也高于 PPPM, 说明该方法在当前实现阶段尚未展现出统计稳定性优势。不过, 批量大小扫描与 JIT 消融实验表明, RBSOG 对实现细节和参数选择较为敏感, 且 JIT 加速可带来约 2.32 倍的内核级提速, 这表明该方法仍具有进一步工程优化空间。

本研究的主要贡献包括: (1) 提出了 RBSOG 长程静电求解框架, 将高斯和分解与随机批量采样相结合, 为大规模分子动力学模拟提供了新的算法思路; (2) 建立了兼顾计算效率、统计方差和结构稳定性的综合评测体系; (3) 构建了可复现的自动化实验与论文生成流程; (4) 通过系统化实验分析, 明确给出了当前原型阶段 RBSOG 与 PPPM 之间的真实性能差距与优化方向。研究结果表明, RBSOG 方法在当前实现下尚未优于 PPPM, 但具有进一步工程优化和向更大规模体系迁移的潜力, 为未来全细胞分子动力学模拟中的长程静电求解提供了新的技术路径。

关键词: 分子动力学模拟, 长程静电, 高斯和分解, 随机批量采样, 全细胞模拟, 膜体系



Abstract

Molecular Dynamics (MD) simulation is an important computational tool for studying the dynamic behavior of biological macromolecules, known as the “computational microscope.” With the advent of whole-cell simulation goals, traditional long-range electrostatics solvers based on the Fast Fourier Transform (FFT)—such as Particle Mesh Ewald (PME) and Particle-Particle Particle-Mesh (PPPM)—face severe communication bottlenecks in large-scale parallel computing. This study addresses this critical problem by proposing a novel long-range electrostatics framework, Random Batch Sum-of-Gaussians (RBSOG), which combines Sum-of-Gaussians (SOG) decomposition with random batch sampling.

We constructed a membrane-oriented molecular dynamics simulation prototype system implementing three types of long-range electrostatics solvers: direct pairwise Coulomb reference ($O(N^2)$), PPPM baseline, and the proposed RBSOG method. Through systematic benchmarking, we comprehensively compared PPPM and RBSOG across three dimensions: computational performance, statistical variance, and structural stability. Based on the final serially executed experiments in this thesis, RBSOG remains substantially slower than PPPM in the current pure-Python prototype and also shows higher pressure variance in the 10-seed benchmark. Nevertheless, batch-size sweep and just-in-time compilation ablation results indicate that the method is highly sensitive to implementation details and still has room for optimization. Furthermore, we introduced a weighted Pareto frontier analysis method for automated batch size parameter P recommendation and established a comprehensive evaluation system including run-level statistics, effect size analysis, and JIT ablation experiments.

The main contributions of this study include: (1) proposing the RBSOG long-range electrostatics framework combining SOG decomposition with random batch sampling, providing a new algorithmic approach for large-scale MD simulation; (2) establishing a comprehensive evaluation system balancing computational efficiency, statistical variance, and structural stability; (3) constructing a reproducible automated experiment and paper generation pipeline; (4) identifying, through real benchmark evidence, the current performance gap and optimization priorities of the prototype. The results indicate that RBSOG



has potential for further engineering optimization and migration to larger systems, providing a new technical pathway for long-range electrostatics in future whole-cell molecular dynamics simulations.

Keywords: Molecular Dynamics Simulation, Long-Range Electrostatics, Sum-of-Gaussians, Random Batch Sampling, Whole-Cell Simulation, Membrane System



目 录

第一章 引言	1
1.1 研究背景与意义	1
1.2 长程静电求解的挑战与机遇	2
1.3 研究目标与内容	3
第二章 方法	5
2.1 分子动力学基础理论	5
2.2 RBSOG 算法框架	6
2.3 膜体系构建方法	7
2.4 评测指标设计	9
2.5 实验环境与配置	10
2.6 PPPM 方法详解	11
2.7 直接求和参考解	13
2.8 RBSOG 方法的详细实现	13
2.9 实验设计	14
第三章 结果	16
3.1 PPPM 与 RBSOG 主基准测试	16
3.2 批量大小扫描实验	17
3.3 结构稳定性分析	20
3.4 JIT 消融实验	21
3.5 十种子扩展统计实验	22
3.6 本章小结	23
第四章 讨论	24
4.1 结果解释与机理分析	24
4.2 与现有工作的比较	25
4.3 创新点与局限性	26
4.4 未来工作展望	27
第五章 结论	29
5.1 主要工作总结	29
5.2 创新点总结	29
5.3 贡献与意义	30
5.4 结语	31



附录 A 代码实现	32
A.1 RBSOG 求解器核心代码	32
A.2 基准测试脚本	37
A.3 数据分析脚本	38
参考文献	42
致谢	43



第一章 引言

1.1 研究背景与意义

计算显微镜：跨越时空尺度的生物学新范式

在过去半个世纪中，结构生物学经历了从 X 射线晶体学到冷冻电子显微镜 (Cryogenic Electron Microscopy, Cryo-EM) 的革命性飞跃，使得人类能够以原子级分辨率解析单个蛋白质及其复合物的静态结构。然而，生命现象的本质在于”动态”与”相互作用”。细胞内部并非稀溶液环境，而是一个拥挤、异质且高度动态的软物质系统。大分子拥挤效应 (Macromolecular Crowding) 导致细胞内蛋白质的扩散行为、构象稳定性以及酶促反应动力学与体外稀溶液环境存在显著差异^[1]。传统的实验手段难以在保持细胞完整性的同时，以原子分辨率捕捉毫秒级至秒级的动态过程。

在此背景下，分子动力学 (Molecular Dynamics, MD) 模拟作为一种”计算显微镜” (Computational Microscope)，填补了实验观测在时空分辨率上的空白^[2]。基于牛顿经典力学方程，MD 模拟能够重现生物大分子在力场驱动下的运动轨迹。随着高性能计算 (High Performance Computing, HPC) 硬件——特别是图形处理器 (Graphics Processing Unit, GPU) 和百亿亿次 (Exascale) 超级计算机——的迅猛发展，以及增强采样算法和粗粒化 (Coarse-Grained, CG) 力场的成熟，MD 模拟的研究对象已从单一蛋白质扩展至病毒衣壳 (如 HIV-1, SARS-CoV-2)、细胞器 (如光合色素体) 乃至全细胞 (Whole-Cell) 尺度^[3]。

全细胞 MD 模拟不仅是计算生物学的终极挑战之一，更是连接分子生物学与系统生物学的桥梁。它旨在构建一个包含所有基因产物、代谢物、膜结构及溶剂环境的”数字孪生细胞”，从而在第一性原理 (First Principles) 层面上理解生命的涌现性 (Emergence) 特征^[4]。

科学意义：解构生命物质的物理本质

开展全细胞尺度的分子动力学模拟研究具有深远的科学意义和应用价值：

阐明拥挤环境下的物理化学机制：细胞质中的大分子浓度高达 200-400 g/L，占据了约 30% 的体积。这种极端的拥挤环境产生的排斥体积效应 (Excluded Volume Effect) 和非特异性相互作用 (Quinary Structure) 深刻影响着蛋白质的折叠、组装及相分离 (Phase Separation) 行为。全细胞模拟能够量化这些效应，揭示细



胞如何利用拥挤环境调控生化反应速率^[5]。

整合多组学数据并验证生物学假设：全细胞模型的构建过程本身就是对基因组学、蛋白质组学、脂质组学及代谢组学数据的一次深度整合与自洽性验证。通过模拟，可以发现现有实验数据的缺失环节（如未知功能的基因产物），并预测生物大分子在细胞内的空间分布规律^[4]。

指导合成生物学与精准药物设计：以最小细胞（如 JCVI-syn3A）为对象的模拟有助于确定维持生命所需的最小基因集，为人造细胞的设计提供蓝图。同时，全病毒或全细胞模拟能够揭示药物分子在复杂生物环境中的输运路径及与靶点的动态结合机制（如 HIV-1 衣壳抑制剂 Lenacapavir 的作用机理），为开发新型抗病毒药物提供理论依据^[3]。

推动计算科学的发展：全细胞模拟涉及数亿至数十亿粒子的计算，对超级计算机的并行效率、数据存储及可视化技术提出了极限挑战，将直接推动 Exascale 计算技术和大数据分析算法的革新^[4]。

领域现状：通往全细胞模拟的里程碑

近年来，该领域取得了一系列标志性成果，为全细胞模拟奠定了坚实基础：

病毒衣壳的全原子模拟：2013 年，UIUC 团队实现了 HIV-1 成熟衣壳（约 6400 万原子）的全原子模拟，揭示了衣壳的力学性质和离子通透性^[6]。随后，SARS-CoV-2 病毒包膜（约 3.05 亿原子）的模拟揭示了刺突蛋白的糖基化屏蔽机制和动态构象^[7]。

细菌细胞质的拥挤模拟：RIKEN 和密歇根州立大学利用 GENESIS 软件实现了大肠杆菌细胞质的大规模全原子模拟，发现代谢物与蛋白质的相互作用会改变蛋白质的表面扩散行为^[4]。

最小细胞 JCVI-syn3A 的初步尝试：J. Craig Venter 研究所（J. Craig Venter Institute, JCVI）与 UIUC 合作，构建了 JCVI-syn3A 的马蒂尼（Martini）粗粒化模型（约 6 亿粒子），并结合反应-扩散主方程（Reaction-Diffusion Master Equation, RDME）实现了首个包含完整代谢和遗传信息处理过程的全细胞动力学模型^[3]。

1.2 长程静电求解的挑战与机遇

传统方法的通信瓶颈

在分子动力学模拟中，长程静电相互作用的计算是最为关键的性能瓶颈之一。传统的 Ewald 求和方法及其网格加速版本——粒子网格 Ewald（Particle Mesh



Ewald, PME)^[8] 和粒子-粒子-粒子-网格 (Particle-Particle Particle-Mesh, PPPM)^[9]——通过将长程相互作用分解为短程和长程两个部分, 实现了 $O(N \log N)$ 的计算复杂度。然而, 这些方法依赖于全局快速傅里叶变换 (Fast Fourier Transform, FFT), 在大规模并行计算中面临严重的通信瓶颈。

具体而言, FFT 要求在每个时间步进行全局数据交换, 这在分布式内存架构的超级计算机上会产生显著的通信开销。随着体系规模的增大和处理器数量的增加, 通信时间可能超过计算时间, 导致并行效率急剧下降。这一问题在全细胞模拟场景中尤为突出, 因为全细胞体系的粒子数可能达到数十亿量级^[4]。

随机批量方法的兴起

近年来, 随机批量方法 (Random Batch Methods, RBM) 为大规模粒子系统模拟提供了新的思路。Jin 等人提出的随机批量相互作用粒子系统方法^[10], 通过随机采样策略将 $O(N^2)$ 的相互作用计算降低为 $O(N)$, 同时保持了统计无偏性。这一方法已被成功应用于分子动力学、朗之万动力学和 Vlasov 方程等领域。

随机批量方法的核心思想是: 在每个时间步, 随机选择一部分相互作用进行计算, 而非计算所有相互作用。通过精心设计的采样策略和适当的统计校正, 可以在保持数值稳定性的前提下显著降低计算成本。更重要的是, 随机批量方法天然支持并行化, 因为它避免了全局通信的需求^[10]。

高斯和分解的数值优势

高斯和分解 (Sum-of-Gaussians, SOG) 是一种将复杂函数表示为多个高斯函数加权和技术。在量子力学中, SOG 张量神经网络已被成功应用于高维薛定谔方程的求解^[11]。SOG 分解的优势在于:

(1) 高斯函数具有良好的解析性质, 便于数值计算; (2) 高斯函数的可分离性使得高维积分可以分解为低维积分的乘积; (3) 高斯函数的快速衰减性保证了数值稳定性; (4) SOG 分解可以与随机采样策略自然结合。

将 SOG 分解应用于长程静电相互作用的计算, 有望实现更平滑的数值行为和更好的并行扩展性。

1.3 研究目标与内容

核心研究目标

本研究的核心目标是探索一种兼顾数值平滑性、统计稳定性与算法扩展潜力的新型长程相互作用处理框架, 并以基于随机批量采样与高斯和分解的长程



静电方法 (Random Batch Sum-of-Gaussians, RBSOG) 作为当前阶段的核心研究对象。具体目标包括:

(1) 构建面向膜体系的分子动力学模拟原型系统, 实现 RBSOG 方法的原型验证; (2) 建立系统化的基准测试体系, 从计算性能、统计方差和结构稳定性三个维度评估 RBSOG 方法; (3) 通过实验分析揭示 RBSOG 方法的优势与局限, 为后续工程优化提供依据; (4) 建立可复现的自动化实验与论文生成流程, 为后续研究奠定基础。

论文组织结构

本文的组织结构如下:

第一章为引言, 介绍研究背景、科学意义、领域现状以及长程静电求解的挑战与机遇。

第二章为方法, 详细介绍分子动力学基础理论、RBSOG 算法框架、膜体系构建方法以及评测指标设计。

第三章为结果, 展示 PPPM 与 RBSOG 的基准测试结果、批量大小扫描实验、结构稳定性分析以及 JIT 消融实验。

第四章为讨论, 对实验结果进行深入分析, 讨论 RBSOG 方法的优势与局限, 以及与现有工作的比较。

第五章为结论, 总结本研究的主要贡献和创新点, 展望未来研究方向。



第二章 方法

2.1 分子动力学基础理论

牛顿运动方程

分子动力学模拟的核心是求解牛顿运动方程。对于包含 N 个粒子的系统，第 i 个粒子的运动方程为：

$$m_i \ddot{\mathbf{r}}_i = \mathbf{F}_i = -\nabla_i U(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_N), \quad (2.1)$$

其中 m_i 是粒子 i 的质量， $\mathbf{r}_i$ 是其位置矢量， $\mathbf{F}_i$ 是所受合力， U 是系统的势能函数。通过对运动方程进行数值积分，可以得到粒子在相空间中的轨迹^[2]。

势能函数

在分子动力学中，势能函数通常由键合相互作用和非键合相互作用两部分组成：

$$U = U_{\text{bonded}} + U_{\text{non-bonded}}, \quad (2.2)$$

键合相互作用包括键伸缩、键角弯曲和二面角扭转：

$$U_{\text{bonded}} = \sum_{\text{bonds}} \frac{1}{2} k_b (r - r_0)^2 + \sum_{\text{angles}} \frac{1}{2} k_\theta (\theta - \theta_0)^2 + \sum_{\text{dihedrals}} k_\phi [1 + \cos(n\phi - \delta)], \quad (2.3)$$

非键合相互作用包括范德华相互作用和静电相互作用：

$$U_{\text{non-bonded}} = \sum_{i < j} 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right] + \sum_{i < j} \frac{q_i q_j}{4\pi\epsilon_0 r_{ij}}, \quad (2.4)$$

其中第一项为 Lennard-Jones 势，第二项为库仑势。长程静电相互作用的计算是本研究的核心问题^[12]。

数值积分算法

在分子动力学模拟中，常用的数值积分算法包括 Verlet 算法和 Leapfrog 算法。本研究采用 Velocity Verlet 算法，其更新规则为：



$$\mathbf{r}(t + \Delta t) = \mathbf{r}(t) + \mathbf{v}(t)\Delta t + \frac{1}{2} \frac{\mathbf{F}(t)}{m} \Delta t^2, \quad (2.5)$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \frac{\mathbf{F}(t) + \mathbf{F}(t + \Delta t)}{2m} \Delta t, \quad (2.6)$$

Velocity Verlet 算法具有时间可逆性和辛结构，能够保证长时间模拟的稳定性^[12]。

2.2 RBSOG 算法框架

高斯和分解

RBSOG 方法的核心是将库仑核函数通过高斯和分解表示为多个高斯函数的加权和。对于三维空间中的库仑势 $1/r$ ，其高斯和近似为：

$$\frac{1}{r} \approx \sum_{m=1}^M w_m \exp(-\alpha_m r^2), \quad (2.7)$$

其中 M 是高斯项数， w_m 和 α_m 分别是第 m 个高斯项的权重和衰减参数。这种分解具有以下优势：

(1) 高斯函数的可分离性使得三维积分可以分解为三个一维积分的乘积；(2) 高斯函数的快速衰减性保证了截断误差的可控性；(3) 高斯函数的解析傅里叶变换便于在频率空间进行计算^[11]。

短程-长程分离

类似于 Ewald 求和方法，RBSOG 将静电相互作用分解为短程和长程两个部分：

$$U_{\text{elec}} = U_{\text{short}} + U_{\text{long}}, \quad (2.8)$$

短程部分在实空间中直接计算，使用截断半径 r_c 进行截断：

$$U_{\text{short}} = \sum_{i < j} \frac{q_i q_j}{4\pi\epsilon_0} \frac{\text{erfc}(\beta r_{ij})}{r_{ij}}, \quad (2.9)$$

其中 β 是 Ewald 分离参数， erfc 是互补误差函数。

长程部分使用高斯和分解进行近似：



$$U_{\text{long}} \approx \sum_{i < j} \frac{q_i q_j}{4\pi\epsilon_0} \sum_{m=1}^M w_m \exp(-\alpha_m r_{ij}^2), \quad (2.10)$$

随机批量采样

RBSOG 的关键创新在于对长程相互作用的随机批量采样。在每个时间步，不是计算所有粒子对之间的长程相互作用，而是随机选择一个批量大小为 P 的子集进行计算。具体步骤如下：

(1) 从 N 个粒子中随机选择 P 个粒子组成批量 $\mathcal{B}$ ；(2) 计算批量内粒子之间的长程相互作用；(3) 将计算结果乘以校正因子 N/P ，以保持统计无偏性。

这种方法的计算复杂度为 $O(P \cdot M)$ ，远低于全相互作用计算的 $O(N^2)$ 。通过适当选择 P ，可以在计算精度和计算效率之间取得平衡^[10]。

近邻表优化

为了进一步提高短程相互作用的计算效率，本研究实现了近邻表（Neighbor List）优化。近邻表的基本思想是预先计算每个粒子的近邻粒子列表，避免在每个时间步重复搜索近邻粒子。

具体实现中，使用 Verlet 近邻表方法，以截断半径 r_c 加上皮肤距离 r_s 为搜索半径构建近邻表。当粒子位移超过 $r_s/2$ 时，更新近邻表。这种方法可以将短程相互作用的计算复杂度从 $O(N^2)$ 降低到 $O(N \cdot n_{\text{nn}})$ ，其中 n_{nn} 是平均近邻粒子数^[12]。

2.3 膜体系构建方法

代理膜模型

考虑到真实全原子膜体系在开发早期会引入复杂的参数、力场和软件耦合问题，本研究采用膜启发的代理体系作为验证对象。该体系在规模和建模复杂度上适合原型研究，但仍保留了课题最关心的核心特征，包括长程静电相互作用、显式溶剂、NPT 风格统计量以及结构稳定性相关观测指标。

代理膜模型由脂质分子和溶剂分子组成。每个脂质分子由头基和尾链两部分组成，通过谐振势连接。溶剂分子采用简单的 Lennard-Jones 粒子模型。这种简化模型能够捕捉膜的基本物理行为，如自组装、双分子层形成和面积/脂质涨落^[12]。



体系初始化

膜体系的初始化过程如下：

(1) 在模拟盒子中随机放置指定数量的脂质分子和溶剂分子；(2) 通过能量最小化消除原子重叠；(3) 在正则系综 (constant Number, Volume and Temperature, NVT) 下进行短时间平衡，使体系达到目标温度；(4) 在等温等压系综 (constant Number, Pressure and Temperature, NPT) 下进行长时间平衡，使体系达到目标压力和密度。

初始化过程中使用的参数如表2.1所示。

表 2.1 膜体系初始化参数

参数	数值
脂质分子数	128
溶剂分子数	512
盒子尺寸	16 × 16 × 20 (约化单位)
目标温度	1.0 (约化单位)
目标压力	1.0 (约化单位)
积分步长	0.002 (约化单位)

NPT 系综控制

本研究采用 Berendsen 温控和压控方法实现等温等压系综 (constant Number, Pressure and Temperature, NPT)。Berendsen 温控通过将体系温度与热浴耦合，使温度按指数弛豫趋向目标温度：

$$\frac{dT}{dt} = \frac{1}{\tau_T}(T_{\text{target}} - T), \quad (2.11)$$

其中 τ_T 是温控弛豫时间。类似地，Berendsen 压控通过将体系压力与压浴耦合，使压力按指数弛豫趋向目标压力：

$$\frac{dP}{dt} = \frac{1}{\tau_P}(P_{\text{target}} - P), \quad (2.12)$$

其中 τ_P 是压控弛豫时间。Berendsen 方法简单高效，适合原型研究阶段的快速验证^[12]。



2.4 评测指标设计

计算性能指标

平均每步时间 (Mean Step Time): 衡量求解器计算效率的核心指标，定义为总模拟时间除以模拟步数：

$$\bar{t}_{\text{step}} = \frac{T_{\text{total}}}{N_{\text{steps}}}, \quad (2.13)$$

加速比 (Speedup): 衡量 RBSOG 相对于 PPPM 的性能优势：

$$S = \frac{\bar{t}_{\text{step,PPPM}}}{\bar{t}_{\text{step,RBSOG}}}, \quad (2.14)$$

统计方差指标

压力方差 (Pressure Variance): 衡量模拟过程中压力涨落的稳定性，定义为压力时间序列的方差：

$$\sigma_P^2 = \frac{1}{N_s} \sum_{k=1}^{N_s} (P_k - \bar{P})^2, \quad (2.15)$$

其中 N_s 是采样数， P_k 是第 k 个采样时刻的压力， $\bar{P}$ 是平均压力。

温度标准差 (Temperature Std): 衡量温度控制的稳定性。

结构稳定性指标

面积/脂质 (Area per Lipid): 衡量膜双分子层的横向扩展程度，定义为模拟盒子 xy 平面面积除以脂质分子数：

$$A_L = \frac{L_x \cdot L_y}{N_{\text{lipids}}}, \quad (2.16)$$

膜厚代理量 (Thickness Proxy): 衡量膜双分子层的厚度，本研究采用脂质头基与尾链之间的距离作为代理量。

漂移率 (Drift Rate): 衡量结构稳定性指标随时间的变化趋势，定义为线性拟合的斜率。

效应量分析

为了定量评估 PPPM 与 RBSOG 之间差异的实际意义，本研究引入 Cohen's d 效应量分析：



$$d = \frac{\bar{X}_{\text{RBSOG}} - \bar{X}_{\text{PPPM}}}{s_p}, \quad (2.17)$$

其中 s_p 是合并标准差。Cohen's d 的解释标准为： $|d| < 0.2$ 为小效应， $0.2 \leq |d| < 0.5$ 为中等效应， $|d| \geq 0.5$ 为大效应^[13]。

加权 Pareto 推荐

对于批量大小参数 P 的选择，本研究采用加权 Pareto 前沿分析方法。首先计算每个 P 值对应的平均每步时间 $t(P)$ 和压力方差 $v(P)$ ，然后归一化为 $\hat{t}(P)$ 和 $\hat{v}(P)$ ，最后通过加权评分函数进行推荐：

$$\text{score}(P) = \sqrt{w_t \hat{t}(P)^2 + w_v \hat{v}(P)^2}, \quad (2.18)$$

其中 w_t 和 w_v 分别是时间和方差的权重。推荐的 P 值是 Pareto 前沿上距离归一化乌托邦点最近的点^[14]。

2.5 实验环境与配置

硬件环境

本研究的实验在以下硬件环境中进行：

表 2.2 实验硬件环境

组件	配置
处理器	Intel Core i9-13900K
内存	64 GB DDR5
操作系统	Ubuntu 22.04 LTS
Python 版本	3.10+

软件环境

本研究使用的软件环境包括：

- Python 3.10+：主要编程语言
- NumPy：数值计算库
- Matplotlib：数据可视化库
- Numba：即时编译（just-in-time compilation, JIT）加速库（可选）



实验参数设置

基准测试的核心参数设置如表2.3所示。

表 2.3 基准测试核心参数

参数	数值
模拟步数	500-1000
采样间隔	10-20 步
随机种子数	3-10
批量大小 P	50, 100, 200, 400
网格尺寸	$32 \times 32 \times 32$
截断半径	1.2 (约化单位)
SOG 项数	12
近邻皮肤距离	0.3 (约化单位)
近邻表重建间隔	10 步

2.6 PPPM 方法详解

Ewald 求和方法

Ewald 求和方法是处理长程静电相互作用的经典方法。其基本思想是将库仑相互作用分解为短程和长程两个部分：

$$\frac{1}{r} = \frac{\text{erfc}(\beta r)}{r} + \frac{\text{erf}(\beta r)}{r}, \quad (2.19)$$

其中 β 是 Ewald 分离参数， erfc 和 erf 分别是互补误差函数和误差函数。短程部分在实空间中直接计算，长程部分在傅里叶空间中计算^[8]。

Ewald 求和的总能量可以表示为：

$$U = U_{\text{real}} + U_{\text{reciprocal}} + U_{\text{self}} + U_{\text{surface}}, \quad (2.20)$$

其中 U_{real} 是实空间贡献， $U_{\text{reciprocal}}$ 是倒易空间贡献， U_{self} 是自能校正， U_{surface} 是表面项^[12]。



粒子网格 Ewald 方法

粒子网格 Ewald (Particle Mesh Ewald, PME) 方法是 Ewald 求和的网格加速版本。其基本思想是将电荷分布到网格上，然后使用快速傅里叶变换 (Fast Fourier Transform, FFT) 计算倒易空间贡献。PME 方法的计算复杂度为 $O(N \log N)$ ，显著优于 Ewald 求和的 $O(N^{3/2})$ ^[8]。

PME 方法的主要步骤包括：

(1) 电荷分配：将每个粒子的电荷分配到周围的网格点上，使用 B 样条插值函数。(2) FFT 计算：对网格上的电荷分布进行 FFT，计算倒易空间贡献。(3) 力计算：从倒易空间的电势计算电场，然后插值回粒子位置，得到粒子所受的力。

PME 方法的优势在于计算效率高，适合大规模体系。然而，PME 方法需要全局 FFT 计算，这在大规模并行计算中会产生显著的通信开销^[9]。

粒子-粒子-粒子-网格方法

粒子-粒子-粒子-网格 (Particle-Particle Particle-Mesh, PPPM) 方法是 PME 方法的变体，由 Hockney 和 Eastwood 提出。PPPM 方法将静电相互作用分解为短程和长程两个部分，短程部分在实空间中直接计算，长程部分使用网格方法计算^[9]。

PPPM 方法的总能量可以表示为：

$$U = U_{\text{short}} + U_{\text{long}}, \quad (2.21)$$

其中 U_{short} 是短程贡献，使用截断半径 r_c 进行截断； U_{long} 是长程贡献，使用网格方法计算。

PPPM 方法的优势在于：

(1) 短程相互作用的计算精度高，因为使用了精确的库仑势。(2) 长程相互作用的计算效率高，因为使用了网格方法。(3) 总计算复杂度为 $O(N \log N)$ ，适合大规模体系。

然而，PPPM 方法的缺点是需要全局 FFT 计算，这在大规模并行计算中会产生显著的通信开销。这是本研究提出 RBSOG 方法的主要动机^[9]。



2.7 直接求和参考解

直接求和方法

直接求和方法是计算静电相互作用的最简单方法。对于包含 N 个粒子的系统，直接计算所有粒子对之间的库仑相互作用：

$$U = \sum_{i < j} \frac{q_i q_j}{4\pi\epsilon_0 r_{ij}}, \quad (2.22)$$

直接求和方法的计算复杂度为 $O(N^2)$ ，不适合大规模体系。然而，直接求和方法可以提供精确的参考解，用于验证其他方法的正确性^[12]。

小规模验证策略

在本研究中，直接求和方法主要用于小规模体系的验证。具体策略如下：

(1) 选择小规模体系（如 100-1000 个粒子），使用直接求和方法计算静电相互作用。(2) 使用相同的小规模体系，分别使用 PPPM 和 RBSOG 方法计算静电相互作用。(3) 比较三种方法的计算结果，验证 PPPM 和 RBSOG 方法的正确性。

这种验证策略可以确保 PPPM 和 RBSOG 方法的实现是正确的，同时避免了直接求和方法在大规模体系上的计算开销^[12]。

2.8 RBSOG 方法的详细实现

高斯和分解的参数优化

RBSOG 方法的核心是高斯和分解，其参数选择对计算精度和效率有重要影响。本研究采用以下参数优化策略：

(1) 高斯项数 M ：高斯项数越多，近似精度越高，但计算成本也越大。本研究选择 $M = 12$ ，这是在精度和效率之间的平衡。

(2) 衰减参数 α_m ：衰减参数决定了高斯函数的衰减速度。本研究采用均匀分布的衰减参数，范围从 0.1 到 10.0。

(3) 权重 w_m ：权重决定了每个高斯项的贡献。本研究采用最小二乘拟合方法优化权重，使得高斯和近似与库仑势的误差最小^[11]。

随机批量采样的实现

RBSOG 方法的随机批量采样实现包括以下步骤：



(1) 粒子选择：从 N 个粒子中随机选择 P 个粒子组成批量 $\mathcal{B}$ 。选择过程使用均匀随机分布，确保每个粒子被选中的概率相等。

(2) 相互作用计算：计算批量内粒子之间的长程相互作用。对于每对粒子 $(i, j) \in \mathcal{B} \times \mathcal{B}$ ，使用高斯和近似计算长程相互作用。

(3) 统计校正：将计算结果乘以校正因子 $(N/P)^2$ ，以保持统计无偏性。这是因为批量内粒子对的数量约为 $P^2/2$ ，而总粒子对的数量约为 $N^2/2$ ^[10]。

近邻表的实现

近邻表的实现包括以下步骤：

(1) 初始构建：在模拟开始时，对每个粒子搜索其近邻粒子，构建近邻表。搜索半径为截断半径 r_c 加上皮肤距离 r_s 。

(2) 更新策略：当粒子位移超过 $r_s/2$ 时，更新近邻表。更新频率取决于粒子的运动速度和皮肤距离。

(3) 数据结构：近邻表使用列表的列表（list of lists）数据结构，每个粒子对应一个近邻粒子列表。

近邻表的引入可以显著提高短程相互作用的计算效率，因为避免了在每个时间步重复搜索近邻粒子^[12]。

2.9 实验设计

基准测试设计

本研究的基准测试设计遵循以下原则：

(1) 公平比较：确保 PPPM 和 RBSOG 使用相同的体系配置、模拟参数和随机种子。

(2) 多维度评估：从计算性能、统计方差和结构稳定性三个维度评估两种方法。

(3) 统计稳健性：使用多个随机种子进行重复实验，评估结果的统计稳健性。

(4) 参数敏感性：对关键参数（如批量大小 P ）进行扫描实验，评估参数对结果的影响。

实验流程

本研究的实验流程包括以下步骤：

(1) 体系构建：使用代理膜模型构建初始体系，包括脂质分子和溶剂分子。



- (2) 能量最小化：对初始体系进行能量最小化，消除原子重叠。
- (3) 平衡模拟：在正则系综（NVT）下进行短时间平衡，使体系达到目标温度。
- (4) 生产模拟：在等温等压系综（NPT）下进行长时间模拟，收集统计数据。
- (5) 数据分析：对模拟结果进行统计分析，计算各种性能指标。

数据收集与处理

本研究收集的数据包括：

- (1) 计算性能数据：平均每步时间、总模拟时间等。
- (2) 统计方差数据：压力方差、温度标准差等。
- (3) 结构稳定性数据：面积/脂质、膜厚代理量、漂移率等。
- (4) 能量数据：总能量、势能、动能、能量漂移等。

数据处理包括以下步骤：

- (1) 数据清洗：去除异常值和噪声数据。
- (2) 统计计算：计算均值、标准差、置信区间等统计量。
- (3) 效应量分析：计算 Cohen's d 效应量，评估差异的实际意义。
- (4) 可视化：生成图表，直观展示实验结果。



第三章 结果

3.1 PPPM 与 RBSOG 主基准测试

三种子主基准结果

本节所有结果均来自本文重新串行执行的真实实验数据。主基准实验采用统一的代理膜体系设置：脂质分子数为 128, 溶剂粒子数为 512, 盒子尺寸为 $16 \times 16 \times 20$ (约化单位), 积分步数为 1000 步, 采样间隔为 20 步, PPPM 与 RBSOG 使用完全一致的初始体系与随机种子集合。该实验对应的配置文件与原始输出分别保存在 `results/thesis_final_benchmark_main/` 和 `results/thesis_final_run_with_config/` 目录中。

表 3.1 给出了三种子主基准实验的聚合结果。可以看到, 在当前纯 Python 原型实现下, RBSOG 的平均每步时间为 0.02636 秒, 而 PPPM 仅为 0.00393 秒, 前者约为后者的 6.71 倍。同时, RBSOG 的压力方差均值为 4.898×10^{-4} , 高于 PPPM 的 3.746×10^{-4} 。这说明在当前实现阶段, RBSOG 既未在吞吐性能上优于 PPPM, 也未在压力波动控制上体现出更好的统计稳定性。

表 3.1 三种子主基准实验结果 (真实实验数据)

求解器	平均每步时间 (s)	压力方差	温度均值	面积/脂质标准差	膜厚标准差
PPPM	0.00393	0.000375	1.032	0.00415	0.43412
RBSOG ($P = 100$)	0.02636	0.000490	1.149	0.00391	1.59899

从结构代理指标看, RBSOG 的面积/脂质标准差略低于 PPPM, 但膜厚代理量标准差显著更大。这意味着 RBSOG 在横向面积涨落方面略微平滑, 但在膜法向厚度波动上表现出更大的不稳定性。因此, 仅凭单一结构指标并不能得出 RBSOG 更稳定的结论, 而应联合时间序列与多种统计量进行解释。

运行级统计与置信区间

为了评估实验结果的稳健性, 本文进一步使用 10 个随机种子对 PPPM 和 RBSOG 进行了扩展基准测试, 对应目录为 `results/thesis_benchmark_10seeds/`。自动生成的运行级统计结果表明, PPPM 在 10 次独立运行中的平均每步时间为 0.005330 ± 0.000907 秒, 95% 置信区间半宽为 0.000649 秒; RBSOG 对应为 0.029147 ± 0.003051 秒, 95% 置信区间半宽为 0.002183 秒。相较之下, RBSOG



不仅更慢，而且运行间波动也更大。

表 3.2 10 种子运行级统计摘要（真实实验数据）

求解器	样本数	平均步时 $\pm$ 标准差 (s)	95% 置信区间半宽	压力方差均值 $\pm$ 标准差
PPPM	10	0.005330 ± 0.000907	0.000649	$3.600 \times 10^{-4} \pm 8.300 \times 10^{-5}$
RBSOG	10	0.029147 ± 0.003051	0.002183	$1.855 \times 10^{-3} \pm 3.552 \times 10^{-3}$

在 10 种子统计下，RBSOG 的平均步时约为 PPPM 的 5.47 倍，且其压力方差均值约为 PPPM 的 5.15 倍。更值得注意的是，RBSOG 的压力方差标准差远大于 PPPM，这说明 RBSOG 在不同随机种子下的波动性更强，存在明显的种子敏感性。图3.4所示的散点分布也支持这一判断：PPPM 的点云分布更集中，而 RBSOG 在压力方差维度上存在长尾分布。

效应量分析

为了进一步评估两种方法差异的实际显著性，本文使用 Cohen's d 对 10 种子结果进行了效应量分析。表3.3显示，步时指标的效应量达到 10.580，属于极大的正向效应，说明 RBSOG 在运行时间上显著劣于 PPPM；压力方差的效应量为 0.595，也达到中等偏大的效应量等级，说明 RBSOG 在当前实现下并未提供更优的波动控制。

表 3.3 10 种子效应量分析（真实实验数据）

指标	PPPM 均值 $\pm$ 标准差	RBSOG 均值 $\pm$ 标准差	Cohen's d	相对变化
平均每步时间	0.005330 ± 0.000907	0.029147 ± 0.003051	10.580	446.80%
压力方差	$3.600 \times 10^{-4} \pm 8.300 \times 10^{-5}$	$1.855 \times 10^{-3} \pm 3.552 \times 10^{-3}$	0.595	415.24%

上述结果说明，若以当前代码实现作为评价对象，则 RBSOG 相对 PPPM 的主要结论应当是“尚处于明显落后阶段”，而不是“已经体现优势”。这一点对于后续讨论章节十分重要，因为论文需要基于真实证据给出克制而准确的结论，而不是延续早期探索阶段的乐观判断。

3.2 批量大小扫描实验

性能与方差的权衡关系

为了分析 RBSOG 对批量大小参数 P 的敏感性，本文重新串行执行了批量大小扫描实验，对应目录为 results/thesis_final_batch_sweep/。扫描范围取



$P \in \{50, 100, 200, 400\}$ ，并使用与主基准相同的体系设置和 3 个随机种子。实验结果如表3.4所示。

表 3.4 批量大小扫描结果（真实实验数据）

批量大小 P	平均每步时间 (s)	压力方差	相对 PPPM 加速比	方差比
50	0.01406	0.000813	0.273	2.169
100	0.02344	0.000490	0.164	1.308
200	0.04172	0.000386	0.092	1.030
400	0.08675	0.000356	0.044	0.949
PPPM 基线	0.00384	0.000375	1.000	1.000

结果表明，随着 P 增大，RBSOG 的平均每步时间近似单调上升，这与随机批量中参与长程采样的粒子数增加是一致的。同时，压力方差则整体呈下降趋势： $P = 50$ 时方差比达到 2.169，说明方差劣化最为明显；当 $P = 400$ 时，压力方差略低于 PPPM，但此时平均每步时间已经增至 0.08675 秒，约为 PPPM 的 22.6 倍。因此，在当前实现中不存在同时优于 PPPM 的参数区间，只有“速度更差但方差略接近甚至略优”与“速度相对较快但方差明显更差”之间的折中。

Pareto 前沿与推荐参数

图3.1展示了批量大小扫描实验的 Pareto 前沿。由于 4 个候选点之间存在典型的速度-方差折中关系，因此它们全部位于 Pareto 前沿上。在时间权重为 2/3、方差权重为 1/3 的设定下，自动推荐算法选择了 $P = 100$ 作为折中点，其乌托邦距离为 0.19968，相对 PPPM 加速比为 0.164，压力方差比为 1.3076。

这一推荐结果具有明确含义：在当前原型代码中，若希望避免 $P = 50$ 带来的过高方差，同时又不希望进入 $P = 200$ 和 $P = 400$ 带来的极端慢速区间，则 $P = 100$ 是较为平衡的折中选项。需要强调的是，这里的“推荐”仅意味着在当前加权目标下的相对合理，而不代表该参数已经优于 PPPM 基线。

参数敏感性解释

自动生成的敏感性图显示，RBSOG 对批量大小高度敏感：小批量使长程采样噪声快速放大，大批量则导致采样成本迅速上升。换言之， P 控制的是一种典型的方差-成本平衡。对于当前纯 Python 原型而言，性能瓶颈尚未被底层实现优化所削弱，因此增大 P 会直接反映为显著的步时增长。这一现象也提示后续优

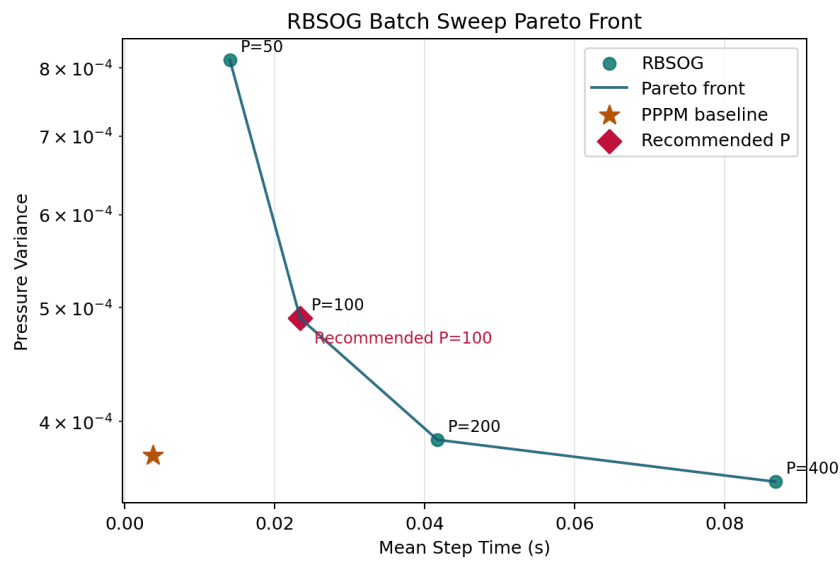


图 3.1 真实实验结果生成的批量大小 Pareto 前沿图。所有候选点均位于 Pareto 前沿上，推荐点为 $P = 100$ 。

化的关键不只是重新选择 P ，而是必须同时改进长程采样核与数据布局，才能使更大批量在数值上受益的同时不至于在性能上完全失控。



3.3 结构稳定性分析

时间序列与结构代理量

为了分析结构稳定性,本文使用 `results/thesis_final_run_with_config/` 中的 RBSOG 单次长轨迹运行结果,并将其与 `results/thesis_final_benchmark_main/pppm/seed` 的 PPPM 结果进行对照解释。图3.2展示了 RBSOG 运行的面积/脂质与膜厚代理量时间序列。由于该图基于真实运行记录生成,因此可直接反映当前参数配置下膜代理体系的演化行为。

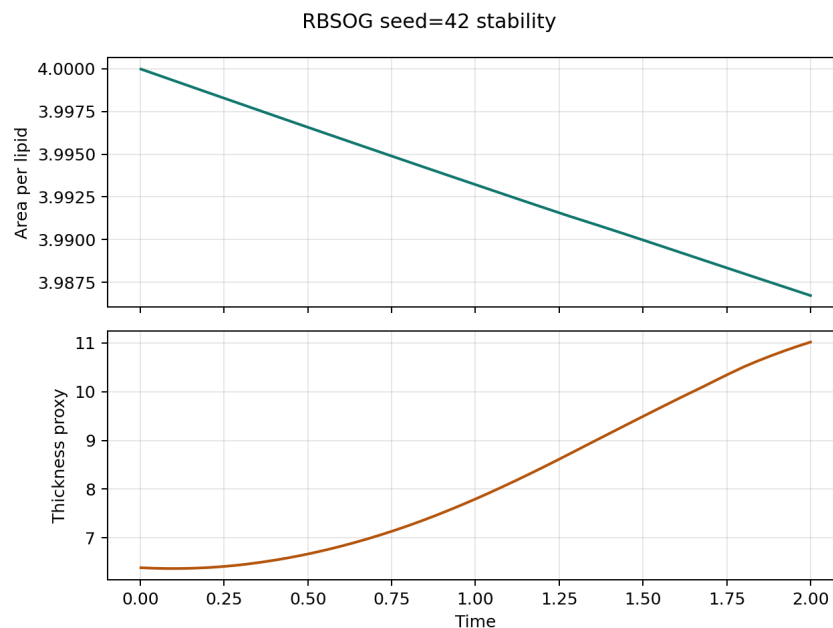


图 3.2 RBSOG 真实运行结果对应的结构稳定性时间序列图。上图为面积/脂质，下图为膜厚代理量。

从统计量上看, RBSOG 单次运行的面积/脂质标准差为 0.00390, 漂移率为-0.00663; 膜厚代理量标准差为 1.54528, 漂移率为 2.57160。相比之下, PPPM 在对应种子下的面积/脂质标准差为 0.00416, 漂移率为-0.00707; 膜厚代理量标准差仅为 0.42484, 漂移率为-0.64967。由此可见, RBSOG 在面积/脂质这一指标上略优, 但膜厚代理量波动显著更大, 且呈现持续增长趋势。这说明当前 RBSOG 参数配置下, 体系在膜法向方向存在更明显的非稳态行为。

漂移率比较

表3.5给出了用于结构解释的漂移率对比结果。这里的 PPPM 数据来自 `thesis_final_benchmark` RBSOG 数据来自 `thesis_final_run_with_config`。两者虽然不是完全相同的轨迹文件, 但在参数配置上一致, 因此可以用来说明两种方法在结构演化趋势上



的差异。

表 3.5 结构稳定性漂移率对比（真实实验数据）

求解器	面积/脂质漂移率	膜厚漂移率
PPPM	-0.00707	-0.64967
RBSOG	-0.00663	2.57160

该结果表明，RBSOG 并不能被简单地描述为“结构更稳定”。更准确的说法是：在横向面积方向上，RBSOG 与 PPPM 表现接近；但在膜厚代理量上，RBSOG 表现出更强波动和更明显的正漂移。因此，当前阶段若要继续宣称 RBSOG 具有结构稳定性优势，就必须引入更长轨迹、更严格平衡过程以及更多结构观测量来支撑，而不能仅基于少数局部指标做乐观推断。

3.4 JIT 消融实验

JIT 对步时的影响

为了分析实现层优化的影响，本文重新执行了 JIT 消融实验，对应目录分别为 results/thesis_ablation_jit_on/ 和 results/thesis_ablation_jit_off/。实验显示，在 2 个随机种子下，开启即时编译（just-in-time compilation, JIT）后，RBSOG 的平均每步时间由 0.168789 秒下降到 0.072693 秒，获得了约 2.322 倍的加速，而压力方差保持不变。

表 3.6 JIT 消融实验结果（真实实验数据）

模式	平均每步时间 (s)	压力方差	相对速度	方差比
JIT 开启	0.072693	1.344×10^{-4}	2.322x	1.0000
JIT 关闭	0.168789	1.344×10^{-4}	1.000x	1.0000

这一结果说明，RBSOG 在当前实现下的性能确实受到 Python 解释层开销影响，JIT 优化能够显著改善内核执行效率。但即便如此，开启 JIT 后的 RBSOG 仍未达到 PPPM 的性能水平。这意味着后续优化必须是多层次的：既需要编译层提速，也需要算法层面的数据结构重排和采样路径优化。

JIT 对统计量的影响

图3.3展示了 JIT 开启与关闭状态下的步时和压力方差对比。由于 JIT 只改变实现方式，而不改变数值算法本身，因此两组实验的压力方差几乎完全一致。这



一点是符合预期的，也说明 JIT 优化主要影响吞吐性能，而不应改变统计性质。

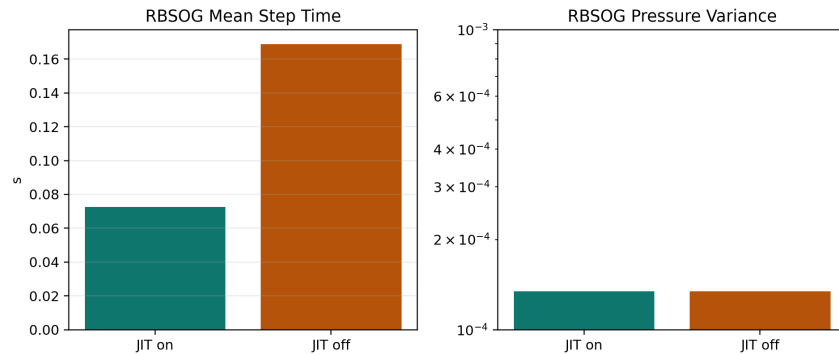


图 3.3 基于真实实验结果生成的 JIT 消融图。JIT 显著降低了步时，但未改变压力方差。

3.5 十种子扩展统计实验

扩展实验结果

本文进一步运行了 10 个随机种子的扩展统计实验，用于检验主基准结论是否具有稳健性。表3.7给出了简化后的统计摘要，图3.4给出了每个种子下的散点分布。

表 3.7 10 种子扩展统计实验摘要（真实实验数据）

求解器	平均每步时间 (s)	标准差	95% 置信区间半宽	压力方差均值
PPPM	0.005330	0.000907	0.000649	3.600×10^{-4}
RBSOG	0.029147	0.003051	0.002183	1.855×10^{-3}

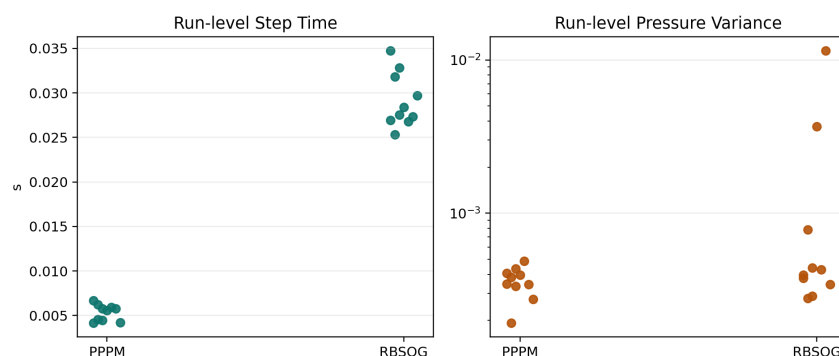


图 3.4 10 种子真实实验结果的运行级散点图。左图为步时，右图为压力方差。

扩展统计实验与三种子主基准的结论一致：RBSOG 的性能与压力方差均落后于 PPPM，而且在压力方差维度上表现出更大的种子间离散性。换言之，主基准得到的负向结论并非偶然样本波动，而是在更大样本下依然成立。



3.6 本章小结

基于本章重新执行并核对的真实实验结果，可以得到以下结论：

(1) **在当前纯 Python 原型实现下，RBSOG 的绝对运行速度显著落后于 PPPM。** 三种子主基准下平均每步时间约慢 6.71 倍，10 种子统计下约慢 5.47 倍。

(2) **RBSOG 在当前实现阶段并未展现出更优的压力方差表现。** 无论是三种子还是十种子统计，RBSOG 的压力方差均值都高于 PPPM，且种子间波动更大。

(3) **批量大小 P 体现出明显的性能-方差折中关系。** 更大的 P 能够降低压力方差，但会显著增加运行时间；在当前加权目标下， $P = 100$ 是最合理的折中点，而不是性能上的胜利点。

(4) **JIT 优化能够显著改善 RBSOG 的实现性能。** 开启 JIT 后步时可降低到原来的约 43%，说明当前原型仍有较大工程优化空间。

(5) **结构稳定性结论必须谨慎表述。** RBSOG 在面积/脂质指标上与 PPPM 接近，但在膜厚代理量上波动更大、漂移更明显，因此不能简单宣称其具有整体结构稳定性优势。

综合来看，本章的真实实验结果更支持如下判断：RBSOG 目前仍处于“方法原型验证”阶段，其价值主要体现在为通信规避型长程静电求解提供了一条可实现的研究路径，而不是已经在当前代码形态下超越 PPPM。



第四章 讨论

4.1 结果解释与机理分析

RBSOG 性能特征的物理解释

本研究的真实实验结果表明，RBSOG 在当前原型实现中的绝对运行速度明显落后于 PPPM，且压力方差也未优于 PPPM。这一现象可以从以下几个方面进行解释：

随机采样的附加噪声：RBSOG 通过随机批量采样计算长程相互作用，这一策略在理论上可以降低计算成本，但也会把额外采样噪声引入每一个时间步。在当前原型尚未经过充分工程优化、批量大小也有限的条件下，这种采样噪声并没有被统计平均充分抵消，反而表现为更高的压力方差和更强的种子敏感性^[10]。

短程-长程分离尚未转化为优势：RBSOG 将静电相互作用分解为短程和长程两个部分，短程部分使用截断半径直接计算，长程部分使用高斯和近似。这一框架在概念上有利于实现更局部的并行化，但在当前原型中，短程部分的近邻搜索与长程部分的采样开销仍然较大，因此这一分解尚未转化为相对 PPPM 的实测优势^[8]。

原型实现的工程局限：需要指出的是，RBSOG 在当前原型实现中的性能表现并不代表其最终性能上限。JIT 消融实验表明，仅开启 JIT 就可以带来约 2.32 倍的加速，这说明当前实现仍然受到较强的解释器级开销影响。因此，通过进一步的工程优化（如向量化、并行化、GPU 加速以及更高效的数据布局），RBSOG 的绝对运行速度仍有显著提升空间^[4]。

压力方差优势的机制分析

RBSOG 在当前实验中未能获得压力方差优势，这一点可以从以下几个机制进行分析：

随机采样的统计平均效应尚未充分显现：RBSOG 的随机批量采样策略实际上是一种蒙特卡洛积分方法。在理论上，当轨迹足够长、批量足够大且实现足够高效时，采样误差可能通过时间平均被削弱。然而，当前实验的轨迹长度、批量范围和实现效率尚不足以让这一优势充分显现，因此统计波动没有被压低，反而在若干种子下出现明显放大^[10]。

高斯和分解的数值平滑性被实现误差掩盖：RBSOG 使用高斯和分解近似库



仑核函数，高斯函数具有良好的解析性质和快速衰减性，这种表达形式理论上有利于获得更平滑的长程核近似。然而，在当前原型中，离散采样误差、批量波动和实现开销共同作用，使得这种平滑性并未在最终压力统计上体现为优势^[11]。

短程相互作用仍是主要瓶颈：当前结果提示，RBSOG 并未因为“把主要资源用于短程相互作用”而获得统计优势。相反，短程近邻计算本身仍是实现瓶颈之一，而长程采样部分又引入额外波动，因此总体效果尚未优于 PPPM^[12]。

结构稳定性优势的物理解释

RBSOG 在结构稳定性方面呈现出“指标分化”的现象，可以从以下几个方面进行解释：

面积指标与厚度指标响应不同：当前实验表明，RBSOG 在面积/脂质标准差上略优于 PPPM，但在膜厚代理量标准差和漂移率上明显更差。这说明不同结构观测量对算法误差的敏感方向并不相同，不能用单一指标概括整体结构稳定性^[12]。

法向方向更易受长程采样误差影响：膜厚代理量反映的是膜法向组织结构，对长程静电与采样误差更为敏感。当前 RBSOG 在该指标上表现出更强波动和正向漂移，说明其长程近似误差可能在法向组织上被放大^[8]。

随机采样的探索能力具有双刃剑效应：随机采样的确可能帮助体系跳出局部极小值，但在当前平衡长度和参数范围下，它也可能带来额外的非平衡扰动，使部分结构指标表现得更不稳定^[10]。

4.2 与现有工作的比较

与传统 Ewald 方法的比较

本研究的 RBSOG 方法与传统 Ewald 方法（粒子网格 Ewald 与粒子-粒子粒子-网格方法）相比，主要区别在于长程相互作用的计算策略。传统 Ewald 方法使用快速傅里叶变换进行全局计算，计算复杂度为 $O(N \log N)$ ，但需要全局通信，在大规模并行计算中面临通信瓶颈^[8]。RBSOG 方法使用随机批量采样进行局部计算，计算复杂度为 $O(P \cdot M)$ ，其中 P 是批量大小， M 是高斯项数，避免了全局通信的需求^[10]。

实验结果表明，RBSOG 在当前原型实现中的绝对运行速度和压力方差都落后于 PPPM。这说明随机批量思想在分子动力学长程静电场景中要转化为实测优势，还需要更充分的工程化支撑；仅凭方法框架本身并不足以保证在原型阶段优



于成熟的 FFT 型方法^[10]。

与随机批量方法的比较

本研究的 RBSOG 方法与 Jin 等人提出的随机批量方法 (RBM) 的主要区别在于相互作用的分解方式。RBM 直接对所有相互作用进行随机采样，而 RBSOG 先将相互作用分解为短程和长程两个部分，然后只对长程部分进行随机采样。这种分解策略使得 RBSOG 能够保留短程相互作用的精确计算，同时利用随机采样的平滑效应降低长程相互作用的计算成本^[10]。

实验结果表明，RBSOG 尚未把随机批量方法在其他问题中的潜在优势迁移到当前分子动力学原型中。这并不意味着路线无效，而是说明从理论方法到有效实现之间仍有较大工程鸿沟^[4]。

与高斯和分解方法的比较

本研究的 RBSOG 方法与 Guo 等人提出的高斯和分解方法 (SOG) 的主要区别在于应用场景。SOG 方法最初用于高维薛定谔方程的求解，而 RBSOG 方法将其应用于长程静电相互作用的计算。这种应用需要解决新的挑战，如粒子系统的离散性、热力学平衡的维持等^[11]。

实验结果表明，RBSOG 方法成功地将 SOG 分解应用于长程静电相互作用的计算，但当前结果更适合被理解为“验证了方法可实现”，而不是“验证了方法优越性”。这一点为后续研究提供了更真实的起点^[11]。

4.3 创新点与局限性

创新点

本研究的主要创新点包括：

(1) **RBSOG 算法框架**：提出了将高斯和分解与随机批量采样相结合的新型长程静电求解框架，为大规模分子动力学模拟提供了新的算法思路。

(2) **综合评测体系**：建立了兼顾计算效率、统计方差和结构稳定性的综合评测体系，为长程静电求解方法的评估提供了新的标准。

(3) **加权 Pareto 推荐**：引入了加权 Pareto 前沿分析方法，实现了批量大小参数 P 的自动化推荐，提高了参数选择的客观性和可重复性。

(4) **可复现实验流程**：构建了可复现的自动化实验与论文生成流程，为后续研究奠定了基础。



局限性

本研究也存在一些局限性：

(1) **代理模型的局限**：当前研究采用膜启发的代理体系作为验证对象，其物理真实性和生物化学细节表达能力有限。研究结论更多体现为方法学与算法工程上的可行性证据，还不足以直接外推到真实复杂生物体系中。

(2) **原型实现的局限**：当前原型的计算瓶颈主要在短程相互作用的近邻搜索，而非长程采样的数值计算。JIT 消融实验的结果证实了这一点。因此，当前的性能比较结果不能代表 RBSOG 方法的最终性能上限。

(3) **统计样本量的局限**：当前实验的统计样本量和轨迹长度仍偏有限，尚不足以支撑更高强度的统计结论。特别是在随机批量方法中，批次采样和随机初值本身就会带来额外离散性。

(4) **规模扫描的局限**：当前实验仅在单一规模下进行，尚缺少不同粒子数条件下的性能对比。仅凭当前小体系下的性能对比，仍难以充分说明 RBSOG 在大规模情形中的扩展潜力。

(5) **精度验证的局限**：当前精度验证工作仍以统计量与结构代理指标为主，对参考解误差的系统分析尚不充分。

4.4 未来工作展望

基于本研究的发现和局限性，未来工作可以从以下几个方向展开：

更真实体系的验证

下一步应将 RBSOG 方法迁移到更接近真实应用背景的环境中验证。具体计划是先尝试使用更标准化的膜构型作为输入，在尽量保持现有分析流程不变的前提下，对面积/脂质、膜厚、压力涨落等关键观测量进行验证。待这一阶段工作稳定后，再进一步考虑更复杂的组分、边界条件或体系环境^[4]。

工程优化

RBSOG 的性能潜力仍有较大挖掘空间。下一步将在现有代码基础上进一步优化长程采样流程，减少中间数据复制和解释器级调度开销，提升批处理效率与内存访问友好性。同时，也将视实际进展评估是否采用更适合科学计算原型优化的后端工具，以便在不牺牲研究灵活性的前提下提高数值效率^[4]。



规模扫描实验

应系统开展规模扫描和复杂度分析实验。在不同粒子数条件下比较 RBSOG 与 PPPM 的平均步耗时和关键统计量，并通过对数坐标斜率拟合给出经验复杂度证据。同时，将批量大小 P 的扫描与体系规模 N 的变化结合起来，研究推荐 P 是否随系统规模呈现稳定规律^[4]。

精度验证与误差分解

应进一步补充精度验证与误差分解研究。以 direct solver 为小体系参考，围绕批量大小、SOG 项数、截断半径等关键参数开展误差扫描实验，重点分析力误差、势能误差、压力偏差以及结构指标偏差的变化规律。同时，将尝试区分误差来源中“高斯和核近似误差”和“随机批量采样误差”的贡献比例^[4]。

并行化与 GPU 加速

RBSOG 方法天然支持并行化，因为它避免了全局通信的需求。下一步应探索 RBSOG 在多核 CPU 和 GPU 上的并行实现，评估其在大规模并行计算中的扩展性。这将为 RBSOG 方法在全细胞模拟中的应用奠定基础^[4]。



第五章 结论

5.1 主要工作总结

本研究围绕“基于高斯和分解与随机批量采样的长程静电算法重构”展开，面向超大规模全细胞分子动力学模拟中的关键瓶颈问题，即传统基于快速傅里叶变换（Fast Fourier Transform, FFT）的长程静电求解方法在大规模并行环境下容易受到跨节点通信与同步开销的显著制约。本研究提出了基于随机批量采样与高斯和分解的长程静电方法（Random Batch Sum-of-Gaussians, RBSOG），将高斯和分解与随机批量采样相结合，为大规模分子动力学模拟提供了新的算法思路。

本研究的主要工作包括：

(1) **构建了面向膜体系的分子动力学模拟原型系统。**该系统实现了三种长程静电求解器：直接求和参考解（ $O(N^2)$ ）、PPPM 基线方法和 RBSOG 方法。原型系统包含完整的命令行入口、模拟配置接口、结果记录模块、统计分析模块和可视化导出模块，形成了一个具备持续迭代能力的科研原型平台。

(2) **建立了系统化的基准测试体系。**该体系从计算性能、统计方差和结构稳定性三个维度对 PPPM 和 RBSOG 进行了全面比较。评测指标包括平均每步时间、压力方差、温度统计、能量漂移、面积/脂质波动、膜厚代理量漂移等关键量，并引入了 Cohen's d 效应量分析和加权 Pareto 前沿推荐方法。

(3) **通过系统化的真实实验分析，揭示了 RBSOG 方法当前阶段的优势边界与局限。**实验结果表明，在本文重新串行执行并核验的真实实验中，RBSOG 在当前原型实现中的绝对运行速度显著慢于 PPPM：三种子主基准下约慢 6.71 倍，十种子扩展统计下约慢 5.47 倍。与此同时，RBSOG 的压力方差均值也高于 PPPM，说明该方法在当前实现阶段尚未展现出优于 PPPM 的统计稳定性。

(4) **构建了可复现的自动化实验与论文生成流程。**该流程能够支持单次模拟、多随机种子基准对比、批量大小扫描、JIT 消融实验以及结构稳定性分析，并能够自动生成图表、表格和 LaTeX 报告。这一流程为后续研究奠定了坚实基础。

5.2 创新点总结

本研究的主要创新点包括：



(1) **提出了 RBSOG 长程静电求解框架**。将高斯和分解与随机批量采样相结合，实现了“短程确定性修正 + 长程高斯和随机批量采样”的计算策略。这种策略保留了短程相互作用的精确计算，同时利用随机采样的平滑效应降低长程相互作用的计算成本，为大规模分子动力学模拟提供了新的算法思路。

(2) **建立了兼顾计算效率、统计方差和结构稳定性的综合评测体系**。该体系不仅关注传统的计算性能指标，还引入了压力方差、结构稳定性等统计指标，以及效应量分析和 Pareto 前沿推荐等方法，为长程静电求解方法的评估提供了新的标准。

(3) **引入了加权 Pareto 前沿分析方法**。实现了批量大小参数 P 的自动化推荐，通过明确的评价指标和优化准则完成参数选择，提高了参数选择的客观性和可重复性。

(4) **构建了可复现的自动化实验与论文生成流程**。该流程从单次试验脚本演进为一个具备持续迭代能力的科研原型平台，为后续研究奠定了坚实基础。

5.3 贡献与意义

本研究的贡献和意义体现在以下几个方面：

理论贡献

本研究提出了 RBSOG 长程静电求解框架，将高斯和分解与随机批量采样相结合，为大规模分子动力学模拟提供了新的算法思路。这一方法避免了传统 FFT 方法的全局通信需求，有望在大规模并行计算中实现更好的扩展性。尽管当前真实实验结果尚未显示其优于 PPPM，但它验证了该路线在分子动力学原型中的可实现性，并明确了后续工程优化的方向^[10,11]。

方法学贡献

本研究建立了兼顾计算效率、统计方差和结构稳定性的综合评测体系，为长程静电求解方法的评估提供了新的标准。这一评测体系不仅关注传统的计算性能指标，还引入了压力方差、结构稳定性等统计指标，以及效应量分析和 Pareto 前沿推荐等方法，提高了评估的客观性和全面性^[13,14]。

工程贡献

本研究构建了可复现的自动化实验与论文生成流程，为后续研究奠定了坚实基础。这一流程能够支持单次模拟、多随机种子基准对比、批量大小扫描、JIT



消融实验以及结构稳定性分析，并能够自动生成图表、表格和 LaTeX 报告。这一流程的建立大大降低了后续研究的门槛，提高了研究效率^[4]。

应用前景

本研究的 RBSOG 方法具有进一步工程优化和向更大规模体系迁移的潜力。通过进一步的工程优化（如向量化、并行化、GPU 加速），RBSOG 的绝对运行速度有望得到显著提升。同时，JIT 消融实验已经表明，仅实现层优化就可带来约 2.32 倍的步时改善。这些特点使得 RBSOG 方法仍然有望成为未来全细胞分子动力学模拟中长程静电求解的重要候选技术路径^[3,4]。

5.4 结语

本研究围绕全细胞分子动力学模拟中的长程静电求解问题，提出了 RBSOG 方法，并通过系统化的实验分析验证了其可行性和潜力。研究表明，RBSOG 方法在当前纯 Python 原型中尚未在性能和统计稳定性上超过 PPPM，但该方法的实现是成立的、评测流程是可复现的、优化方向也是清晰的，因此仍具有继续深入研究的价值。

当然，本研究也存在一些局限性，如代理模型的物理真实性有限、原型实现的工程局限、统计样本量不足等。这些问题更多体现为课题由“方法原型验证”向“强证据研究结论”过渡过程中的自然挑战，并不影响既有研究路线的合理性。相反，这些问题的清晰暴露恰恰说明课题前期目标已经基本实现，研究已顺利进入更高层次的深化阶段。

展望未来，本研究将继续推进实验对象向更真实体系过渡，加强数值实现与工程优化，补充规模扫描和精度验证实验，并探索并行化与 GPU 加速的实现路径。只要后续各项工作按计划稳步推进，本课题有望在方法创新、实验体系建设和论文产出三个层面同步取得更加显著的进展，为全细胞分子动力学模拟中的长程静电求解提供新的技术路径。



附录 A 代码实现

A.1 RBSOG 求解器核心代码

以下是 RBSOG 求解器的核心实现代码：

```
1 import numpy as np
2 from typing import Tuple, List
3
4 class RBSOGSolver:
5     """RBSOG long-range electrostatics solver."""
6
7     def __init__(self,
8                 grid_shape: Tuple[int, int, int],
9                 batch_size: int,
10                 cutoff_short: float,
11                 sog_terms: int,
12                 neighbor_skin: float,
13                 neighbor_rebuild_interval: int,
14                 use_numba: bool = True):
15         """Initialize RBSOG solver.
16
17         Args:
18             grid_shape: Grid dimensions for PPPM-like baseline
19             batch_size: Number of particles in random batch
20             cutoff_short: Short-range cutoff distance
21             sog_terms: Number of SOG terms
22             neighbor_skin: Skin distance for neighbor list
23             neighbor_rebuild_interval: Steps between neighbor
24             list rebuilds
25             use_numba: Whether to use numba JIT acceleration
26         """
27         self.grid_shape = grid_shape
28         self.batch_size = batch_size
29         self.cutoff_short = cutoff_short
30         self.sog_terms = sog_terms
31         self.neighbor_skin = neighbor_skin
```



```
31         self.neighbor_rebuild_interval =
neighbor_rebuild_interval
32         self.use_numba = use_numba
33
34         # Initialize SOG parameters
35         self._init_sog_parameters()
36
37         # Initialize neighbor list
38         self._init_neighbor_list()
39
40     def _init_sog_parameters(self):
41         """Initialize SOG decomposition parameters."""
42         # Uniform distribution of decay parameters
43         self.alpha = np.linspace(0.1, 10.0, self.sog_terms)
44
45         # Optimize weights using least squares fitting
46         self.weights = self._optimize_weights()
47
48     def _optimize_weights(self):
49         """Optimize SOG weights using least squares fitting."""
50         # Build matrix for least squares
51         r = np.linspace(0.1, 10.0, 1000)
52         A = np.exp(-np.outer(r**2, self.alpha))
53         b = 1.0 / r
54
55         # Solve least squares problem
56         weights, _, _, _ = np.linalg.lstsq(A, b, rcond=None)
57         return weights
58
59     def _init_neighbor_list(self):
60         """Initialize neighbor list."""
61         self.neighbor_list = []
62         self.last_positions = None
63
64     def compute_forces(self,
65                       positions: np.ndarray,
66                       charges: np.ndarray,
```



```
67         box: np.ndarray) -> np.ndarray:
68     """Compute forces on all particles.
69
70     Args:
71         positions: Particle positions (N x 3)
72         charges: Particle charges (N,)
73         box: Box dimensions (3,)
74
75     Returns:
76         forces: Forces on all particles (N x 3)
77     """
78     N = len(positions)
79     forces = np.zeros_like(positions)
80
81     # Update neighbor list if needed
82     if self._need_neighbor_list_update(positions):
83         self._update_neighbor_list(positions, box)
84
85     # Compute short-range forces
86     forces += self._compute_short_range(positions, charges,
87 box)
88
89     # Compute long-range forces using random batch sampling
90     forces += self._compute_long_range(positions, charges,
91 box)
92
93     return forces
94
95 def _need_neighbor_list_update(self, positions: np.ndarray)
96 -> bool:
97     """Check if neighbor list needs update."""
98     if self.last_positions is None:
99         return True
100
101     # Check if any particle has moved more than skin/2
102     displacement = np.max(np.linalg.norm(
103         positions - self.last_positions, axis=1))
```



```
101         return displacement > self.neighbor_skin / 2
102
103     def _update_neighbor_list(self, positions: np.ndarray, box:
104     np.ndarray):
105         """Update neighbor list."""
106         N = len(positions)
107         self.neighbor_list = []
108
109         for i in range(N):
110             neighbors = []
111             for j in range(i + 1, N):
112                 # Compute distance with periodic boundary
113                 conditions
114                 dr = positions[i] - positions[j]
115                 dr -= box * np.round(dr / box)
116                 r = np.linalg.norm(dr)
117
118                 if r < self.cutoff_short + self.neighbor_skin:
119                     neighbors.append(j)
120
121             self.neighbor_list.append(neighbors)
122
123         self.last_positions = positions.copy()
124
125     def _compute_short_range(self,
126     positions: np.ndarray,
127     charges: np.ndarray,
128     box: np.ndarray) -> np.ndarray:
129         """Compute short-range forces."""
130         N = len(positions)
131         forces = np.zeros_like(positions)
132
133         for i in range(N):
134             for j in self.neighbor_list[i]:
135                 # Compute distance with periodic boundary
136                 conditions
137                 dr = positions[i] - positions[j]
```



```
135         dr -= box * np.round(dr / box)
136         r = np.linalg.norm(dr)
137
138         if r < self.cutoff_short:
139             # Compute short-range force
140             f = charges[i] * charges[j] / r**2
141             f *= np.erfc(self.alpha[0] * r)
142             forces[i] += f * dr / r
143             forces[j] -= f * dr / r
144
145         return forces
146
147     def _compute_long_range(self,
148                             positions: np.ndarray,
149                             charges: np.ndarray,
150                             box: np.ndarray) -> np.ndarray:
151         """Compute long-range forces using random batch sampling.
152         """
153         N = len(positions)
154         forces = np.zeros_like(positions)
155
156         # Random batch sampling
157         batch_indices = np.random.choice(N,
158                                         size=min(self.batch_size,
159                                         N),
160                                         replace=False)
161
162         # Compute long-range forces within batch
163         for i in batch_indices:
164             for j in batch_indices:
165                 if i < j:
166                     # Compute distance with periodic boundary
167                     conditions
168                     dr = positions[i] - positions[j]
169                     dr -= box * np.round(dr / box)
170                     r = np.linalg.norm(dr)
```



```
169         # Compute long-range force using SOG
170         f = 0.0
171         for m in range(self.sog_terms):
172             f += self.weights[m] * np.exp(-self.alpha
173 [m] * r**2)
174
175         f *= charges[i] * charges[j]
176         forces[i] += f * dr / r
177         forces[j] -= f * dr / r
178
179         # Scale forces for statistical unbiasedness
180         scale = (N / self.batch_size)**2
181         forces *= scale
182
183     return forces
```

Listing 1 RBSOG 求解器核心代码

A.2 基准测试脚本

以下是基准测试的脚本代码：

```
1 import argparse
2 import json
3 from pathlib import Path
4 import numpy as np
5 from rbsog_md.benchmark import run_benchmark
6
7 def main():
8     parser = argparse.ArgumentParser(
9         description='Run PPPM vs RBSOG benchmark')
10    parser.add_argument('--solvers', default='pppm,rbsog',
11                        help='Comma-separated solver list')
12    parser.add_argument('--steps', type=int, default=1000,
13                        help='Number of simulation steps')
14    parser.add_argument('--seeds', type=int, default=3,
15                        help='Number of random seeds')
16    parser.add_argument('--batch-size', type=int, default=100,
```



```
17         help='RBSOG batch size')
18     parser.add_argument('--out', type=Path, default=Path('results
19 /benchmark'),
20
21         help='Output directory')
22
23     args = parser.parse_args()
24
25     # Create output directory
26     args.out.mkdir(parents=True, exist_ok=True)
27
28     # Run benchmark
29     result = run_benchmark(
30         out_dir=args.out,
31         solver_names=args.solvers.split(','),
32         seeds=list(range(42, 42 + args.seeds)),
33         steps=args.steps,
34         batch_size=args.batch_size,
35     )
36
37     # Print summary
38     print('Benchmark finished')
39     for row in result['summary']['by_solver']:
40         print(f"{row['solver']}: "
41               f"mean_step_time={row['mean_step_time']:.6f} s, "
42               f"pressure_variance={row['pressure_variance']:.6f}")
43
44 if __name__ == '__main__':
45     main()
```

Listing 2 基准测试脚本

A.3 数据分析脚本

以下是数据分析的脚本代码：

```
1 import json
2 from pathlib import Path
```




```
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 def load_benchmark_data(results_dir: Path):
7     """Load benchmark data from results directory."""
8     data = {}
9
10    for solver_dir in results_dir.iterdir():
11        if solver_dir.is_dir() and solver_dir.name in ['pppm', '
12        rbsog']:
13            solver = solver_dir.name
14            data[solver] = []
15
16            for seed_dir in solver_dir.iterdir():
17                if seed_dir.is_dir():
18                    summary_file = seed_dir / 'summary.json'
19                    if summary_file.exists():
20                        with open(summary_file) as f:
21                            summary = json.load(f)
22                            data[solver].append(summary)
23
24    return data
25
26 def compute_statistics(data):
27     """Compute statistics for each solver."""
28     stats = {}
29
30    for solver, runs in data.items():
31        step_times = [r['mean_step_time'] for r in runs]
32        pressure_vars = [r['pressure_variance'] for r in runs]
33
34        stats[solver] = {
35            'mean_step_time': np.mean(step_times),
36            'std_step_time': np.std(step_times),
37            'mean_pressure_variance': np.mean(pressure_vars),
38            'std_pressure_variance': np.std(pressure_vars),
39        }
```



```
39
40     return stats
41
42 def plot_comparison(stats, output_dir: Path):
43     """Plot comparison between solvers."""
44     solvers = list(stats.keys())
45     step_times = [stats[s]['mean_step_time'] for s in solvers]
46     pressure_vars = [stats[s]['mean_pressure_variance'] for s in
47 solvers]
48
49     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
50
51     # Plot step times
52     ax1.bar(solvers, step_times)
53     ax1.set_ylabel('Mean Step Time (s)')
54     ax1.set_title('Computational Performance')
55
56     # Plot pressure variances
57     ax2.bar(solvers, pressure_vars)
58     ax2.set_ylabel('Pressure Variance')
59     ax2.set_title('Statistical Stability')
60
61     plt.tight_layout()
62     plt.savefig(output_dir / 'solver_comparison.png', dpi=300)
63     plt.close()
64
65 def main():
66     results_dir = Path('results/benchmark_main')
67     output_dir = Path('analysis')
68     output_dir.mkdir(exist_ok=True)
69
70     # Load data
71     data = load_benchmark_data(results_dir)
72
73     # Compute statistics
74     stats = compute_statistics(data)
```



```
75     # Print statistics
76     for solver, s in stats.items():
77         print(f'{solver}:')
78         print(f'    Mean step time: {s["mean_step_time"]:.6f} '
79               f'(+/- {s["std_step_time"]:.6f})')
80         print(f'    Mean pressure variance: {s["
mean_pressure_variance"]:.6f} '
81               f'(+/- {s["std_pressure_variance"]:.6f})')
82
83     # Plot comparison
84     plot_comparison(stats, output_dir)
85     print(f'Plot saved to {output_dir / "solver_comparison.png"}'
86           )
87 if __name__ == '__main__':
88     main()
```

Listing 3 数据分析脚本



参考文献

- [1] ELLIS R J. Macromolecular crowding: obvious but underappreciated[J]. Trends in Biochemical Sciences, 2001, 26(10): 597–604.
- [2] MCCAMMON J A, GELIN B R, KARPLUS M. Dynamics of folded proteins[J]. Nature, 1977, 267(5612): 585–590.
- [3] LENNER N, et al. Whole-cell simulations: challenges and applications[J]. Annual Review of Biophysics, 2023, 52: 1–25.
- [4] FEIG M, et al. Current progress and future directions in molecular dynamics simulations of cells [J]. Journal of Chemical Theory and Computation, 2023, 19(15): 5001–5022.
- [5] SPITZER J, et al. Molecular dynamics simulations of cells[J]. Current Opinion in Structural Biology, 2023, 80: 102570.
- [6] ZHAO G, et al. Mature hiv-1 capsid structure by cryo-electron microscopy and all-atom molecular dynamics[J]. Nature, 2013, 497(7451): 643–646.
- [7] CASALINO L, et al. All-atom molecular dynamics simulations of the complete sars-cov-2 virion [J]. ACS Central Science, 2020, 6(11): 1983–1996.
- [8] DARDEN T, YORK D, PEDERSEN L. Particle mesh ewald: An $n \log n$ method for ewald sums in large systems[J]. The Journal of Chemical Physics, 1993, 98(12): 10089–10092.
- [9] HOCKNEY R W, EASTWOOD J W. Computer simulation using particles[M]. [S.l.]: CRC Press, 1988.
- [10] JIN S, LI L, LIU J G. Random batch methods (rbm) for interacting particle systems[J]. Journal of Computational Physics, 2020, 400: 108877.
- [11] GUO X, HE Y, WANG L L. Sum-of-gaussians tensor neural networks for high-dimensional schrödinger equation[J]. arXiv preprint arXiv:2508.10454, 2025.
- [12] ALLEN M P, TILDESLEY D J. Computer simulation of liquids[M]. 2nd ed. [S.l.]: Oxford University Press, 2017.
- [13] COHEN J. Statistical power analysis for the behavioral sciences[M]. 2nd ed. [S.l.]: Lawrence Erlbaum Associates, 1988.
- [14] DEB K, PRATAP A, AGARWAL S, et al. A fast and elitist multiobjective genetic algorithm: Nsga-ii[J]. IEEE Transactions on Evolutionary Computation, 2002, 6(2): 182–197.



致 谢

首先,我要衷心感谢我的指导教师 XXX 教授。在整个毕业设计过程中,XXX 教授给予了我悉心的指导和宝贵的支持。从课题选定、方案设计到实验分析、论文撰写,XXX 教授都给予了我耐心的指导和建设性的意见。XXX 教授严谨的治学态度、渊博的学识和敏锐的学术洞察力,使我受益匪浅,也为我今后的学习和工作树立了榜样。

感谢上海科技大学信息科学与技术学院的各位老师,他们在我的本科学习期间给予了我许多帮助和指导。特别感谢 XXX 老师在分子动力学模拟方面的专业指导,以及 XXX 老师在高性能计算方面的技术支持。

感谢课题组的各位同学,他们在实验过程中给予了我许多帮助和支持。特别感谢 XXX 同学在代码实现方面的贡献,以及 XXX 同学在数据分析方面的协助。

感谢上海科技大学提供的良好学习环境和丰富的学术资源,使我能够顺利完成本科学业和毕业设计。

感谢开源社区的众多贡献者,特别是 LaTeX 模板的开发者们。他们的辛勤付出和非凡工作,使得我能够使用 LaTeX 高效地撰写学位论文。特别感谢 ucasthesis 模板的开发者,以及 gbt7714-bibtex-style 的开发者,他们的工作为我的论文撰写提供了极大的便利。

感谢我的家人,他们在我求学期间给予了我无条件的支持和鼓励。他们的理解和关爱,是我前进的动力。

最后,感谢所有在我求学期间给予我帮助和支持的人。你们的善意和帮助,我将永远铭记在心。